前置知识：

> 弄清楚js、Node.js、js的运行环境有哪些

运行环境：是指代码正常运行所需的必要环境

JS是一门编程语言，Node.js 是js的一种运行环境 Node.js是在**服务器端**运行的**JavaScript运行时环境**，而**客户端中的JS**是在**浏览器**中运行的。

> **客户端（Client）**：发起请求、面向用户的一侧。通常是用户设备上的软件，比如浏览器、手机 App、桌面程序。它负责展示界面、接收用户操作，并向服务器发请求。
>
> **服务器端（Server）**：接收请求、处理业务、返回结果的一侧。它通常运行在服务器机器、云主机、容器里，也可以在你本机跑一个本地服务。它负责数据库访问、权限校验、业务逻辑、接口响应等。

JavaScript的运行环境汇总：

- **浏览器类：** 前端开发

  例如 Chrome浏览器（v8引擎+浏览器内置的各种API 如 DOM/BOM)

  - v8引擎负责解析和执行JavaScript代码。 ps：不同浏览器有不同的引擎
  - 内置API 是由运行环境提供的特殊接口,只能在所属的运行环境中被调用。
- **服务器端/命令行类：** 后端开发

  例如 Node.js（V8 引擎 + Node.js 内置 API，如 `fs`、`http`、`path`、`process`）

  - V8 引擎负责解析和执行 JavaScript 代码。
  - Node.js 内置 API 由 Node.js 运行环境提供，只能在 Node.js 中被调用，例如 `fs.readFile()`、`http.createServer()`、`process.env`。
  - Node.js 环境**没有 DOM/BOM**，不能直接操作网页；但它可以读写文件、创建网络服务器、访问进程信息等。
  - Node.js 还通过 `libuv` 提供事件循环和非阻塞 I/O，使 JavaScript 能处理文件、网络等服务器端任务。
- **其他 JavaScript 宿主环境：例如 Deno、Bun、Electron、小程序、React Native 等**
- **共同规律：JavaScript 运行环境 ≈ JavaScript 引擎 + 宿主环境 API + 事件循环/任务调度等基础设施** ⭐⭐⭐⭐⭐
  - **ECMAScript 核心语法一致**：变量、函数、对象、数组、`Promise`、`class` 等语言核心在不同环境中基本一致。
  - **宿主 API 决定能力边界**：浏览器提供 `document`、`window`、`fetch` 等；Node.js 提供 `fs`、`http`、`process` 等。
  - **内置 API 不能跨环境直接调用**：浏览器里没有 `fs`，Node.js 里没有 `document`。同一段 JS 在浏览器和 Node.js 中表现不同，就是因为运行环境提供的 API 不同。
  - 如果需要跨环境复用代码，通常要靠适配层、打包工具、polyfill 或条件判断来处理环境差异。

JavaScript 是语言核心，运行环境负责“执行语言”并“提供能力”。浏览器和 Node.js 都能运行 JS，但前者提供网页 API，后者提供服务器端/系统级 API，所以同一门语言在不同环境中能做的事不同。

JavaScript 代码本身不能被计算机直接执行，需要一个**JavaScript 引擎（解释器）** 来解析和运行

不同的浏览器使用不同的JavaScript解析引擎:例如：Chrome 浏览器 => V8，Firefox 使用 SpiderMonkey 其中,Chrome浏览器的V8解析引擎性能最好!

一、初识 Node.js

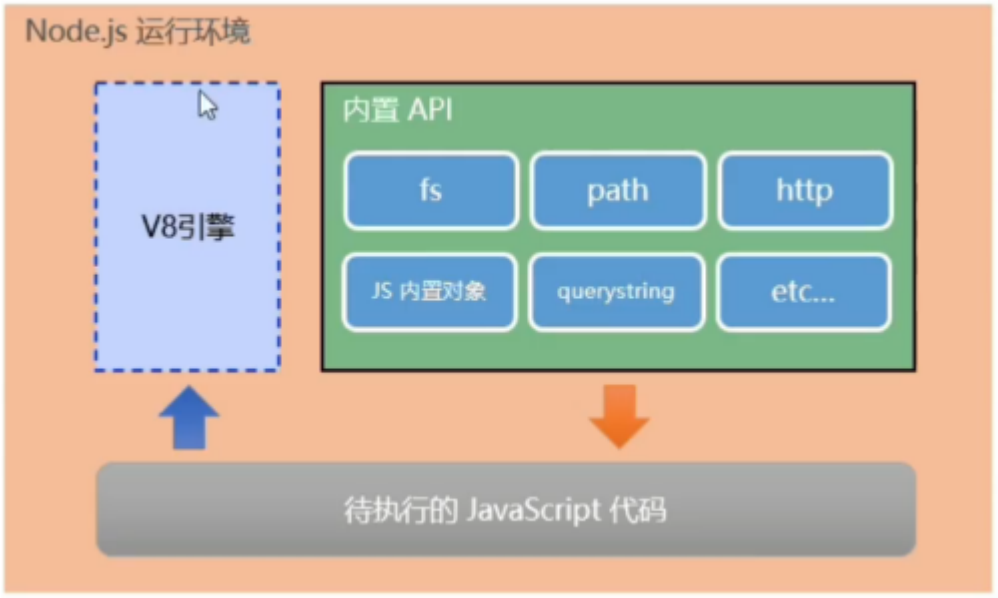
1.什么是node.js

`Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine.`

Node.js 是一个基于Chrome V8引擎的JavaScript运行环境。

Node.js的官网地址:https://nodejs.org/zh-cn/

2.Node.js 中的JavaScript 运行环境



3.Node.js 可以做什么

Node.js作为一个JavaScript的运行环境,仅仅提供了基础的功能和API。然而,基于Node.js提供的这些基础能,很多强大的工具和框架如雨后春笋,层出不穷,所以学会了Node.js,可以让前端程序员胜任更多的工作和岗位:

1 基于Express框架(http://www.expressjs.com.cn/),可以快速构建 Web应用

2 基于Electron框架(https://electronjs.org/),可以构建跨平台的桌面应用

3 基于restify 框架(http://restify.com/),可以快速构建API接口项目

4 读写和操作数据库、创建实用的命令行工具辅助前端开发、etc ...

4.Node.js 学习路径

JavaScript基础语法+Node.js内置API 模块(fs、path、http等)+第三方API模块(express、mysql等)

5.补充：在终端中的快捷键 ⭐⭐⭐⭐⭐⭐⭐⭐⭐⭐

使用↑键,可以快速定位到上一次执行的命令

使用tab键,能够快速补全路径

使用esc键,能够快速清空当前已输入的命令

输入cls命令,可以清空终端

二、fs 文件系统模块

1. 什么是fs文件系统模块

fs 模块是 Node.js官方提供的、用来操作文件的模块。它提供了一系列的方法和属性,用来满足用户对文件的操作需求。

例如:

- `fs.readFile()` 方法,用来读取指定文件中的内容
- `fs.writeFile()` 方法,用来向指定的文件中写入内容

导入fs模块:

```
const fs = require('fs')
```

2.fs.readFile()

```
fs.readFile(path[, options], callback)
```

参数1:必选参数,字符串,表示文件的路径。

参数2:可选参数,表示以什么编码格式来读取文件。

参数3:必选参数,文件读取完成后,通过**回调函数**拿到读取的结果。

示例：以utf8的编码格式,读取指定文件的内容,并打印err和dataStr的值:

```
// 1.导入 fs 模块，来操作文件
const fs = require('fs')

// 2.调用fs.readFile方法，读取文件中的内容
//    参数1：读取文件的存放路径
//    参数2：读取文件时侯采用的编码格式，一般默认指定 utf8
//    参数3：回调函数，用于处理读取文件完成后的结果（拿到读取失败和成功的结果 err dataStr）
fs.readFile('./file/1.txt', 'utf8', (err, dataStr) => {
    console.log(err)  // 打印读取失败的结果
    console.log("-----------------")
    console.log(dataStr)  // 打印读取成功的结果
})
```

如果读取成功，则 err 的值为 null 如果读取失败，则 err 的值为 错误对象，dataStr 的值为 underfined

```
[Error: ENOENT: no such file or directory, open 'D:\实习\实习生全栈学习笔记
\Part1_Node.js\1103\code\file\11.txt'] {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
  path: 'D:\\实习\\实习生全栈学习笔记\\Part1_Node.js\\1103\\code\\file\\11.txt'
}
-----------------
undefined
```

判断文件是否读取成功

```
const fs = require('fs')

fs.readFile('./file/11.txt', 'utf8', (err, dataStr) => {
    if(err) {
       return console.log('文件读取失败'+err.message)
    }
    console.log("文件读取成功" + dataStr)

})
```

3.fs.writeFile()的语法格式

```
fs.writeFile(file, data[, options], callback)
```

参数1:必选参数,需要指定一个文件路径的字符串,表示文件的存放路径。

参数2:必选参数,表示要写入的内容。

参数3:可选参数,表示以什么格式写入文件内容,默认值是utf8。

参数4:必选参数,文件写入完成后的**回调函数。**

示例:

```
// 1.导入 fs 模块，来操作文件
const fs = require('fs')

//2. 调用 fs.writeFile()方法,写入文件的内容
//    参数1:表示文件的存放路径
//    参数2:表示要写入的内容
//    参数3:回调函数

fs.writeFile('./file/2.txt', 'hello node', function(err) {
    if(err) {
       return console.log('文件写入失败'+err.message)
    }
    console.log("文件写入成功")
})
```

如果文件写入成功 err 值等于 null

如果文件写入失败，则 err 的值等于一个 错误对象

4.路径动态拼接的问题

在使用fs模块操作文件时,如果提供的操作路径是以/或 .. /开头的相对路径时,很容易出现路径动态拼接错误的问题。

原因:代码在运行的时候,会以执行node命令时所处的目录,动态拼接出被操作文件的完整路径。

解决方案:在使用fs模块操作文件时,直接提供完整的路径,不要提供./或 .. /开头的相对路径,从而防止路径动态拼接的问题。

提供完成路径的方法：直接在代码中填入 绝对路径；`__dirname` **表示当前代码文件所处的目录**

示例:

```
// __dirname 表示当前文件所处的目录
// console.log(_dirname)
const fs = require('fs')

fs.readFile(__dirname + '/file/1.txt', 'utf8', (err, dataStr) => {
    if(err) {
        return console.log('文件读取失败'+err.message)
    }
    console.log("文件读取成功" + dataStr)
})
```

三、path路径模块

1. 什么是 path 路径模块

path 模块是Node.js官方提供的、用来**处理路径的模块。**它提供了一系列的方法和属性,用来满足用户对路径的处理 需求。

例如:

- path.join()方法,用来将多个路径片段拼接成一个完整的路径字符串
- path.basename()方法,用来从路径字符串中,将文件名解析出来

导入:

```
const path = require('path')
```

2.path.join()的语法格式

使用path.join()方法,可以**把多个路径片段拼接为完整的路径字符串**,语法格式如下:

```
path. join([ ... paths])
```

参数解读:

- ... paths 路径片段的序列
- 返回值 :< string>

示例:

```
const pathStr = path. join('/a', '/b/c',' .. /','./d', 'e')   // ../ 会抵消上一层路径
console.log(pathStr) // \a\b\dle

const pathStr2 = path. join(_dirname, './files/1.txt')
console.log(pathStr2)//输出 当前文件所处目录\files\1.txt
```

注意:今后凡是涉及到路径拼接的操作,都要使用path.join()方法进行处理。不要直接使用+进行字符串的拼接。

```
const fs = require('fs')
const path = require('path')

fs.readFile(path.join(__dirname,'/file/1.txt'), 'utf8', (err, dataStr) => {
    if(err) {
        return console.log('文件读取失败'+err.message)
    }
    console.log("文件读取成功" + dataStr)
})
```

3.path.basename()的语法格式

使用path.basename()方法,**可以获取路径中的最后一部分,经常通过这个方法获取路径中的文件名**,语法格式如下:

```
path.basename(path[, ext])
```

参数解读:

- path必选参数,表示一个路径的字符串
- ext可选参数,表示文件扩展名
- 返回 :< string>表示路径中的最后一部分

示例:

```
const path = require('path')

const fpath='/a/b/c/index.html'// 文件的存放路径

var fullName = path.basename(fpath)
console.log(fullName) // index.html

var nameWithoutExt = path.basename(fpath, '.html')
console.log(nameWithoutExt) // index
```

4.path.extname() 的语法格式

```
path.extname(path)
```

path必选参数,表示一个路径的字符串

示例:

```
const fpath ='/a/b/c/index.html'// 路径字符串

const fext = path. extname(fpath)
console.log(fext)// 输出 .html
```

四、http模块

1.什么是http模块

回顾:什么是客户端、什么是服务器?

🐯 🐯 🐯 在**网络节点**中,负责**消费资源**的电脑,叫做客户端;负责**对外提供网络资源**的电脑,叫做服务器。 🐯 🐯 🐯 🐯

http模块是Node.js官方提供的、用来创建web服务器的模块。通过http模块提供的http.createServer()方法,就 能方便的把一台普通的电脑,变成一台Web服务器,从而对外提供Web资源服务。

```
const http = require('http')
```

2.进一步理解http模块的作用

服务器和普通电脑的区别在于,**服务器上安装了web服务器软件**,例如:IIS、Apache等。通过安装这些服务器软件, 就能把一台普通的电脑变成一台web服务器。

PS:服务器软件—>phpStudy 、 xampp 、 wamp

在Node.js中,我们不需要使用IIS、Apache等这些第三方web服务器软件。因为我们**可以基于Node.js提供的 http模块**,通过几行简单的代码,就能轻松的**手写一个服务器软件,从而对外提供web服务。**

3.服务器相关的概念

3.1IP地址

IP地址就是互联网上每台计算机的唯一地址,因此IP地址具有唯一性。如果把"个人电脑"比作"一台电话",那么"IP地 址"就相当于"电话号码",只有在知道对方IP地址的前提下,才能与对应的电脑之间进行数据通信。

IP地址的格式:通常用"点分十进制"表示成(a.b.c.d)的形式,其中,a,b,c,d都是0~255之间的十进制整数。例如:用 点分十进表示的IP地址(192.168.1.1)

注意:

- **互联网中每台Web服务器,都有自己的IP地址**,例如:大家可以在Windows的终端中运行ping www.baidu.com命令,即可查看到百度服务器的IP地址。
- 在开发期间,自己的电脑既是一台服务器,也是一个客户端,为了方便测试,可以在自己的浏览器中输入**127.0.0.1**这个IP地址,就能把自己的电脑当做一台服务器进行访问了。

3.2 域名和域名服务器

百度的IP地址:183.2.172.177 对应的 域名:www.baidu.com 类比 人类的身份证号(唯一)和姓名

尽管IP地址能够唯一地标记网络上的计算机,但IP地址是一长串数字,不直观,而且不便于记忆,于是人们又发明了另一套**字符型的地址方案**,即所谓的**域名(Domain Name)地址。**

IP地址和域名是一一对应的关系,这份对应关系存放在一种叫做**域名服务器(DNS,Domain name server)**的电脑中。使用者只需通过好记的域名访问对应的服务器即可,对应的转换工作由域名服务器实现。因此,域名服务器就是提供IP地址和域名之间的转换服务的服务器。

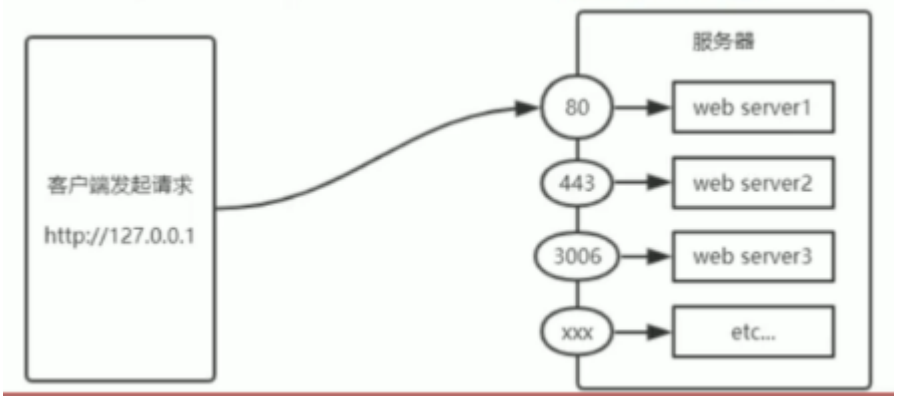
注意:

- 单纯使用IP地址,互联网中的电脑也能够正常工作。但是有了域名的加持,能让互联网的世界变得更加方便。
- 在开发测试期间,**127.0.0.1 对应的域名是localhost**,它们都**代表我们自己的这台电脑**,在使用效果上没有任何区别。

3.3 端口号

计算机中的端口号,就好像是现实生活中的门牌号一样。通过门牌号,外卖小哥可以在整栋大楼众多的房间中,准确把外卖送到你的手中。

同样的道理,在一台电脑中,可以运行成百上千个web服务。**每个web服务都对应一个唯一的端口号。客户端发送过来的网络请求,通过端口号,可以被准确地交给对应的web服务进行处理。**



注意:

1 每个端口号不能同时被多个web服务占用。 2 在实际应用中,URL中的80端口可以被省略。

4.利用http模块创建最基本的web服务器 ⭐⭐⭐⭐⭐

4.1 基本步骤

🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃

- **导入http 模块**
  - ```
    const http = reuiqre('http')
    ```
- **创建 web服务器实例**
  - 调用http.createServer()方法,即可快速创建一个web服务器实例:

    ```
    const server = http.createServer()
    ```
- 为**服务器实例绑定request 事件,监听客户端的请求**
  - 调用服务器实例的 .on()方法，为服务器绑定request 事件
    ```

```
// 使用服务器实例的 ⭐.on()⭐ 方法，为服务器绑定一个 request 事件
server.on('request',(req,res) => {
    // 只要有客户端来请求我们自己的服务器,就会触发 request 事件,从而调用这个事件处理函数
    console.log('Someone visit our web server.')
})
```

- **启动服务器**
  - 调用服务器实例的.listen()方法,即可启动当前的web服务器实例:

```
// 调用 server.listen(端口号,cb回调)方法,即可启动 web 服务器
server.listen(80, () => {
    console.log('http server running at http://127.0.0.1')
})
```

🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃

4.2 req 请求对象

只要服务器接收到了客户端的请求,就会调用通过server.on()为服务器绑定的request 事件处理函数。

如果想在**事件处理函数中,访问与客户端相关的数据或属性**,可以使用如下的方式:

```
server.on('request', (req) => {
    // req 是请求对象,它包含了与客户端相关的数据和属性,例如:
    // req.url 是客户端请求的 URL 地址 ⭐⭐⭐⭐⭐
    // req.method 是客户端的 method 请求类型 ⭐⭐⭐⭐⭐
    const str = 'Your request url is ${req.url}, and request method is ${req.method}
    console.log(str)   // 服务器端（ 自己电脑的终端 显示）
})
```

http://127.0.0.1 —> req.url: / http://127.0.0.1/index.html —> req.url: /index.html

4.3 res 响应对象

在服务器的request事件处理函数中,如果**想访问与服务器相关的数据或属性**,可以使用如下的方式:

```
server.on('request', (req, res) => {
    // res 是响应对象,它包含了与服务器相关的数据和属性,例如:
    // 要发送到客户端的字符串，客户端呈现的内容 ⭐⭐⭐⭐⭐
    const str = 'Your request url is ${req.url}, and request method is ${req.method}'
    // res.end()方法的作用:
    //向客户端发送指定的内容,并结束这次请求的处理过程。即用户端能看到的内容 ⭐⭐⭐⭐⭐
    res.end(str)
})
```

4.4 解决中文乱码的问题

当调用res.end()方法,向客户端发送中文内容的时候,会出现乱码问题,此时,需要手动设置内容的编码格式:

```
server.on('request', (req, res) => {
    //发送的内容包含中文
    const str=`您请求的 url地址是${req.url},请求的 method 类型是${req.method}`
    // 为了防止中文显示乱码的问题,需要设置响应头 Content-Type 的值为 text/html; charset=utf-8
    res. setHeader('Content-Type', 'text/html; charset=utf-8')
    // 把包含中文的内容,响应给客户端
    res.end(str)
})
```

5.根据不同的url响应不同的html内容

5.1 基本步骤

🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃

1 获取**请求的url地址**

2 设置**默认的响应内容**为404 Not found

3 判断用户请求的是否为**/或/index.html 首页**

4 判断用户请求的是否为**/about.html关于页面**

5 设置 **Content-Type响应头**,防止中文乱码

6 使用**res.end() 把内容响应给客户端**

🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃🍃

```
const http = require('http')
const server = http.createServer()

server.on('request', (req, res) => {
  // 1. 获取请求的 url 地址
  const url = req.url
  // 2. 设置默认的响应内容为 404 Not found
  let content = '<h1>404 Not found!</h1>'
  // 3. 判断用户请求的是否为 / 或 /index.html 首页
  // 4. 判断用户请求的是否为 /about.html 关于页面
  if (url === '/' || url === '/index.html') {
    content = '<h1>首页</h1>'
  } else if (url === '/about.html') {
    content = '<h1>关于页面</h1>'
  }
  // 5. 设置 Content-Type 响应头，防止中文乱码
  res.setHeader('Content-Type', 'text/html; charset=utf-8')
  // 6. 使用 res.end() 把内容响应给客户端
  res.end(content)
```

```
})

server.listen(80, () => {
  console.log('server running at http://127.0.0.1')
})
```